BURSA TEKNİK ÜNİVERSİTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

NESNEYE YÖNELİK PROGRAMLAMA

PROJE ÖDEVİ

HAZIRLAYAN:

ANTSANIAINA ZINHA TATIANA TANTELIHARISOA

24360859223

Bilgisayar mühendisliği Bölümü/ Şube 1

DANIŞMAN:

ARŞ. GÖR. ZEYNEP BARUT

ARŞ. GÖR. ŞEYMA DOĞRU

2025-2026

BAHAR DÖNEMİ

İçindekiler

1. GİRİŞ 2

2. TEKNİK DETAYLAR..... 3
    2.1. Program Akışı ve Modüler Yapı 3
    2.2. Sınıf Hiyerarşisi ve Kalıtım 3
    2.3. Sınıfların Detaylı Açıklaması 4
    2.4. Ekran Çıktıları 11

3. EKSİKLİKLER VE GELİŞTİRMELER 12

4. SONUÇ 13

5. KAYNAKÇA 14

GİRİŞ

GitHub Deposu:
https://github.com/Zinha863/NYP_24360859223_ZinhaTanteliharisoa

YouTube Videosu: https://youtu.be/xk4pKx0ZwIM

Bu proje, Bursa Teknik Üniversitesi, Bilgisayar Mühendisliği bölümü, Nesneye Yönelik Programlama dersi kapsamında geliştirilmiş konsol tabanlı bir banka yönetim sistemi uygulamasıdır. Program tamamen Java programlama dili kullanılarak yazılmış olup, herhangi bir grafik arayüz içermemektedir.

Projenin temel amacı, nesneye yönelik programlama prensiplerini kullanarak bir bankanın temel işlemlerini simüle etmektir. Uygulama; müşteri oluşturma, hesap açma, para yatırma, para transferi, kredi kartı tanımlama, borç ödeme ve hesap silme işlemlerini desteklemektedir.

Bu proje bireysel olarak Zinha Tanteliharisoa tarafından geliştirilmiştir. Proje kapsamında öğrenci numaram 24360859223, şubem ise 1'dir.

Programın genel çalışma akışı şu şekildedir: BankaService sınıfı üzerinden müşteri ve banka personeli oluşturulur, müşteriye hesap ve kredi kartı tanımlanır, ardından çeşitli bankacılık işlemleri gerçekleştirilir ve sonuçlar ekrana yazdırılır.

TEKNİK DETAYLAR

2.1 — Program Akışı ve Modüler Yapı

Proje, nesneye yönelik programlama prensiplerine uygun ve modüler bir yapıda geliştirilmiştir. Sınıflar, işlevlerine göre farklı paketler altında organize edilmiştir:

- **com.bank.app.people** paketi: Kisi, Musteri ve BankaPersoneli sınıflarını içermektedir.
- **com.bank.app.accounts** paketi: BankaHesabi, VadesizHesap ve YatirimHesabi sınıflarını içermektedir.
- **com.bank.app.cards** paketi: KrediKarti sınıfını içermektedir.
- **com.bank.app.service** paketi: İş mantığını gerçekleştiren BankaService sınıfını içermektedir.
- **com.bank.app.main** paketi: Uygulamanın başlangıç noktası olan Test sınıfını içermektedir.

Programın genel çalışma akışı şu adımlardan oluşmaktadır:

1. BankaService nesnesi oluşturulur.
2. Müşteri ve banka personeli nesneleri oluşturulur.
3. Müşteri, personelin müşteri listesine eklenir.
4. Müşteri adına vadesiz hesap açılır ve para yatırılır.
5. İkinci bir hesap açılarak hesaplar arası para transferi gerçekleştirilir.
6. Müşteriye kredi kartı tanımlanır ve borç ödeme işlemi yapılır.
7. Bakiyesi olan hesap silinmeye çalışılır ve uyarı mesajı gösterilir.

2.2 — Sınıf Hiyerarşisi ve Kalıtım

Projede kalıtım (inheritance) prensibi uygulanmıştır. Sınıflar arasındaki hiyerarşi aşağıdaki gibidir:

<u>Kişi Hiyerarşisi:</u>

Kisi sınıfı, banka personeli ve müşterilerin ortak özelliklerini barındıran temel sınıftır. Bu sınıftan Musteri ve BankaPersoneli sınıfları türetilmiştir:

```
Kisi
  |---- Musteri
  |---- BankaPersoneli
```

<u>Hesap Hiyerarşisi:</u>

BankaHesabi sınıfı, tüm hesap türlerinin ortak özelliklerini barındıran temel sınıftır. Bu sınıftan VadesizHesap ve YatirimHesabi sınıfları türetilmiştir:

```
BankaHesabi
  |---- VadesizHesap
  |---- YatirimHesabi
```

<u>İlişkiler:</u>

- BankaPersoneli sınıfı, temsil ettiği müşterileri saklamak için Musteri türünde bir ArrayList barındırmaktadır.
- Musteri sınıfı, sahip olduğu hesapları saklamak için BankaHesabi türünde bir ArrayList ve kredi kartlarını saklamak için KrediKarti türünde bir ArrayList barındırmaktadır.

Her sınıfta toString() metodu override edilerek, nesnenin bilgileri anlamlı bir şekilde ekrana yazdırılmaktadır. Alt sınıflardaki toString() metotları, super.toString() çağrısı ile üst sınıfın bilgilerini de içermektedir.

2.3 — Sınıfların Detaylı Açıklaması

<u>Kisi Sınıfı</u>

Kisi sınıfı, banka personeli ve müşterilerin ortak özelliklerini barındıran temel sınıftır. Bu sınıfta ad, soyad, email ve telefonNumarasi alanları bulunmaktadır. Telefon numarası, büyük sayıları desteklemek amacıyla long veri tipi olarak tanımlanmıştır. Tüm alanlar için getter ve setter metotları yazılmıştır.

```java
package com.bank.app.people;

class Kisi {
    private String ad;
    private String soyad;
    private String email;
    private long telefonNumarasi;

    //getter metodlar
    public String getAd() {
        return ad;
    }
    public String getSoyad() {
        return soyad;
    }
    public String getEmail() {
        return email;
    }
    public long getTelefonNumarasi() {
        return telefonNumarasi;
    }

    //setter metodlar
    public void setAd(String ad) {
        this.ad=ad;
    }
    public void setSoyad(String soyad) {
        this.soyad=soyad;
    }
    public void setEmail(String email) {
        this.email=email;
    }
    public void setTelefonNumarasi(long telefonNumarasi) {
        this.telefonNumarasi=telefonNumarasi;
    }

    //Constructor
    public Kisi(String ad ,String soyad ,String email ,long telefonNumarasi){
        this.ad=ad;
        this.soyad=soyad;
        this.email=email;
        this.telefonNumarasi=telefonNumarasi;
    }

    //Ekran çıktısı
    public String toString() {
        return "Ad: "+ad+", soyad: "+soyad+", email: "+email+", telefon numarasi: "+telefonNumarasi;
    }
}
```

Şekil 1: Kisi.java sınıfı kod görüntüsü

<u>Musteri sınıfı</u>

Kisi sınıfından türetilmiştir. musteriNumarasi alanı, Random sınıfı kullanılarak otomatik olarak üretilmektedir. Müşteriye ait hesaplar BankaHesabi türünde, kredi kartları ise KrediKarti türünde ArrayList'lerde saklanmaktadır.

```java
package com.bank.app.people;
import java.util.ArrayList;
import java.util.Random;

import com.bank.app.accounts.BankaHesabi;
import com.bank.app.accounts.VadesizHesap;
import com.bank.app.accounts.YatirimHesabi;
import com.bank.app.cards.KrediKarti;

public class Musteri extends Kisi {
    private String musteriNumarasi = "A" + (new Random().nextInt(900000) + 100000);
    private ArrayList<BankaHesabi> hesaplar= new ArrayList<>();
    private ArrayList<KrediKarti> krediKartlari= new ArrayList<>();

    //Getter metodlar
    public ArrayList<BankaHesabi> getHesaplar(){
        return hesaplar;
    }
    public ArrayList<KrediKarti> getKrediKartlari(){
        return krediKartlari;
    }

    //Constructor
    public Musteri(String ad ,String soyad ,String email ,long telefonNumarasi){
        super(ad,soyad, email, telefonNumarasi);
    }
```

Şekil 2: Musteri.java sınıfı

Bu sınıfta bulunan metotlar şunlardır:

- **hesapEkle:** Parametre olarak aldığı hesap türüne göre VadesizHesap veya YatirimHesabi nesnesi oluşturarak hesaplar listesine ekler.

```java
    public void hesapEkle(String hesapTuru) {
        if (hesapTuru.equals("vadesiz")) {
            hesaplar.add(new VadesizHesap(0));
        }
        else {
            hesaplar.add(new YatirimHesabi(0));
        }
    }

```

Şekil 3: hesapEkle metodu

- **kredikartEkle:** KrediKarti nesnesi oluşturarak krediKartlari listesine ekler.

```java
    public void kredikartEkle(double limit) {
        krediKartlari.add(new KrediKarti(limit,0));
    }

```

Şekil 4: krediKartEkle metodu

- **hesapSil:** Hesabın bakiyesini kontrol eder, bakiye 0'dan büyükse uyarı mesajı gösterir, 0'a eşitse hesabı siler.

```java
    public void hesapSil(BankaHesabi hesap) {
        if (hesap.getBakiye()>0) {
            System.out.println("Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız.");
        }
        else {
            hesaplar.remove(hesap);
        }
    }
```

Şekil 5: hesapSil metodu

- **krediKartSil:** Kartın güncel borcunu kontrol eder, borç 0'dan büyükse uyarı mesajı gösterir, 0'a eşitse kartı siler.

```java
    public void krediKartSil(KrediKarti kart) {
        if (kart.getGuncelBorc()>0) {
            System.out.println("Lütfen öncelikle borç ödemesi yapınız.");
        }
        else {
            krediKartlari.remove(kart);
        }
    }
```

Şekil 6: krediKartSil metodu

BankaPersoneli Sınıfı

BankaPersoneli sınıfı, Kisi sınıfından türetilmiştir. personelID alanı, Random sınıfı kullanılarak otomatik olarak üretilmektedir. Personelin temsil ettiği müşteriler, Musteri türünde bir ArrayList'te saklanmaktadır.

```java
package com.bank.app.people;

import java.util.ArrayList;
import java.util.Random;

public class BankaPersoneli extends Kisi{
    private String personelID= "ID"+ (new Random().nextInt(90000) + 10000);
    private ArrayList<Musteri> musteriler= new ArrayList<>();

    //Getter metod
    public ArrayList<Musteri> getMusteriler(){
        return musteriler;
    }

    //Constructor
    public BankaPersoneli(String ad ,String soyad ,String email ,int telefonNumarasi) {
        super(ad,soyad, email, telefonNumarasi);
    }

    //Ekran çıktısı
    @Override
    public String toString() {
        return super.toString() + ", PersonelID: " + personelID+"\n\n";
    }
}
```

Şekil 7: BankaPersoneli.java sınıfı kod görüntüsü

BankaHesabi Sınıfı

BankaHesabi sınıfı, tüm hesap türlerinin temel sınıfıdır. iban alanı, Random sınıfı kullanılarak "TR" önekiyle otomatik olarak üretilmektedir. bakiye alanı constructor

aracılığıyla başlangıçta 0 olarak atanmaktadır. Tüm alanlar için getter ve setter metotları yazılmıştır.

```java
package com.bank.app.accounts;

import java.util.Random;

public class BankaHesabi {
    private String iban= "TR"+ (new Random().nextInt(900000)+100000);
    private double bakiye;

    //Getter metodlar
    public String getIban() {
        return iban;
    }
    public double getBakiye() {
        return bakiye;
    }

    //Setter metodlar
    public void setIban(String iban) {
        this.iban= iban;
    }
    public void setBakiye(double bakiye) {
        this.bakiye= bakiye;
    }
    public BankaHesabi(double bakiye) {
        this.bakiye= bakiye;
    }

    //Ekran çıktısı
    public String toString() {
        return "IBAN: " + iban + ", Bakiye: " + bakiye;
    }
}
```

Şekil 8: BankaHesabi.java sınıfı kod görüntüsü

<u>VadesizHesap Sınıfı</u>

VadesizHesap sınıfı, BankaHesabi sınıfından türetilmiştir. hesapTuru alanı "vadesiz" olarak sabit tutulmuştur.

```java
package com.bank.app.accounts;

import com.bank.app.cards.KrediKarti;

public class VadesizHesap extends BankaHesabi {
    private String hesapTuru="vadesiz";

    //Getter metod
    public String getHesapTuru() {
        return hesapTuru;
    }

    //Constructor
    public VadesizHesap(double bakiye) {
        super(bakiye);
    }
```

Şekil 9: VadesizHesap sınıfı kod görüntüsü

Bu sınıfta bulunan metotlar şunlardır:

- **paraTransferi:** Alıcı hesap, gönderen hesap ve miktar bilgilerini parametre olarak alır. Gönderen hesabın bakiyesinden miktarı düşer, alıcı hesabın bakiyesine ekler.

```
18    public void paraTransferi(BankaHesabi aliciHesap, BankaHesabi gonderenHesap, double miktar) {
19        gonderenHesap.setBakiye(gonderenHesap.getBakiye() - miktar);
20        aliciHesap.setBakiye(aliciHesap.getBakiye() + miktar);
21    }
```

Şekil 10: paraTransferi metodu

- **krediKartiBorcOdeme:** Kart bilgisi ve ödenecek miktar bilgilerini parametre olarak alır. Hesabın bakiyesinden miktarı düşer ve kredi kartının güncel borcunu azaltır.

```
23    public void krediKartiBorcOdeme(KrediKarti kart, double miktar) {
24        this.setBakiye(this.getBakiye() - miktar);
25        kart.setGuncelBorc(kart.getGuncelBorc() - miktar);
26    }
```

Şekil 11: krediKartBorcOdeme metodu

<u>YatirimHesabi Sınıfı</u>

YatirimHesabi sınıfı, BankaHesabi sınıfından türetilmiştir. hesapTuru alanı "yatirim" olarak sabit tutulmuştur.

```
1  package com.bank.app.accounts;
2
3  public class YatirimHesabi extends BankaHesabi{
4      private String hesapTuru="yatirim";
5
6      //Getter metod
7      public String getHesapTuru() {
8          return hesapTuru;
9      }
10
11     //Constructor
12     public YatirimHesabi(double bakiye) {
13         super(bakiye);
14     }
```

Şekil 12: YatırımHesabi sınıfı kod görüntüsü

Bu sınıfta bulunan metotlar şunlardır:

- **paraEkle:** Belirtilen miktarı hesabın bakiyesine ekler.

```
16    public void paraEkle(double miktar) {
17        this.setBakiye(this.getBakiye() + miktar);
18    }
```

Şekil 13: paraEkle metodu

- **paraCek:** Hesabın bakiyesini kontrol eder, bakiye yetersizse uyarı mesajı gösterir, yeterliyse miktarı bakiyeden düşer.

```java
    public void paraCek(double miktar) {
        if (this.getBakiye()<miktar) {
            System.out.println("Yetersiz bakiye!");
        }
        else {
            this.setBakiye(this.getBakiye() - miktar);
        }
    }
```

Şekil 14: paraCek metodu

KrediKarti Sınıfı

KrediKarti sınıfı, bir müşterinin kredi kartı bilgilerini saklar. kartNumarasi alanı Random sınıfı kullanılarak otomatik olarak üretilmektedir. limit, guncelBorc ve kullanilabilirLimit alanları bulunmaktadır.

```java
package com.bank.app.cards;

import java.util.Random;

public class KrediKarti {
    private String kartNumarasi= String.valueOf(new Random().nextInt(900000)+ 100000);
    private double limit;
    private double guncelBorc;
    private double kullanilabilirLimit;
```

Şekil 15: KrediKart Sınıfı kod görüntüsü

kullanilabilirLimit, toString() metodunda limit ile guncelBorc arasındaki fark olarak hesaplanmaktadır.

```java
    //Ekran çıktısı
    @Override
    public String toString() {
        return "Kart Numarasi: " + kartNumarasi + ", Limit: " + limit + ", Guncel Borc: " + guncelBorc + ", Kullanilabilir Limit: " + (limit - guncelBorc)+"\n";
    }
```

Şekil 16: toString metodu

BankaService Sınıfı

BankaService sınıfı, uygulamanın iş mantığını gerçekleştiren servis sınıfıdır. Tüm bankacılık işlemleri bu sınıf üzerinden yürütülmektedir. Bu sınıfta bulunan metotlar şunlardır:

- **musteriOlustur:** Yeni bir Musteri nesnesi oluşturur ve döndürür.

```java
    //Müşetri olusturmak
    public Musteri musteriOlustur(String ad ,String soyad ,String email ,long telefonNumarasi) {
        return new Musteri(ad,soyad,email,telefonNumarasi);
    }
```

Şekil 17: musteriOlustur metodu

- **hesapAc:** Müşteri adına hesap açar.

```java
    //Müsteri adına Hesap açmak
    public void hesapAc(Musteri musteri, String hesapTuru) {
        musteri.hesapEkle(hesapTuru);
    }
```

Şekil 18: hesapAc metodu

- **paraYatir:** Belirtilen hesaba para yatırır.

```java
    //Hesaba Para yatırmak
    public void paraYatir(BankaHesabi hesap, double miktar) {
        hesap.setBakiye(hesap.getBakiye()+miktar);
    }
```

Şekil 19: paraYatir metodu

- **paraTransferi:** İki hesap arasında para transferi gerçekleştirir.

```java
    //Hesaplar arasında para transferi gerceklestirmek
    public void paraTransferi(VadesizHesap gonderenHesap, BankaHesabi aliciHesap, double miktar) {
        gonderenHesap.paraTransferi(aliciHesap, gonderenHesap, miktar);
    }
```

Şekil 20: paraTransferi metodu

- **krediKartTanimla:** Müşteriye kredi kartı tanımlar.

```java
    //Müsteriye kredi kartı tanımlama
    public void krediKartTanimla(Musteri musteri, double limit) {
        musteri.krediKartEkle(limit);
    }
```

Şekil 21: krediKartTanimla metodu

- **borcOde:** Kredi kartı borcu ödemesi yapar.

```java
    //Kredi kartı borcu ödeme islemi yapmak
    public void borcOde(VadesizHesap hesap, KrediKarti kart, double miktar) {
        hesap.krediKartiBorcOdeme(kart, miktar);
    }
```

Şekil 22: borcOde metodu

- **hesapSil:** Gerekli şartlar sağlandığında hesabı siler.

```java
    //Hesap silmek
    public void hesapSil(Musteri musteri, BankaHesabi hesap) {
        musteri.hesapSil(hesap);
    }
```

Şekil 23: hesapSil metodu

Test Sınıfı

Test sınıfı, uygulamanın başlangıç noktasıdır. Bu sınıfta BankaService nesnesi üzerinden tüm bankacılık işlemleri test edilmektedir. Gerçekleştirilen işlemler sırasıyla şunlardır:

- Müşteri oluşturulur ve ekrana yazdırılır.

```java
        //Müsteri olusturmak
        System.out.println("=== Musteri Olusturuluyor ===");
        Musteri musteri=bankaService.musteriOlustur("Zinha", "Tanteliharisoa", "zinhaantsaharisoa@gmail.com", 05055357147L);
        System.out.println(musteri);
```

Şekil 24: Test.java - Müşteri oluşturma bloğu

- Banka personeli oluşturulur ve müşteri personelin listesine eklenir.

```java
        // Banka personeli olusturmak
        System.out.println("=== Banka Personeli Olusturuluyor ===");
        BankaPersoneli personel = new BankaPersoneli("Ahmet", "Kaya", "ahmet@btu.edu.tr", 5551234);
        System.out.println(personel);

        // Personelin musterisini eklemek
        personel.getMusteriler().add(musteri);
```

Şekil 25: Test.java - Banka personeli oluşturma bloğu

- Müşteri adına vadesiz hesap açılır ve 1000 TL yatırılır.

```java
        //Hesap acmak
        System.out.println("=== Hesap Aciliyor ===\n");
        bankaService.hesapAc(musteri, "vadesiz");

        //Hesaba para yatırmak
        System.out.println("=== Para Yatiriliyor ===");
        VadesizHesap hesap=(VadesizHesap) musteri.getHesaplar().get(0);
        bankaService.paraYatir(hesap, 1000);
        System.out.println(hesap);
```

Şekil 26: Test.java - Hesap açma ve para yatırma bloğu

- İkinci bir vadesiz hesap açılarak birinci hesaptan ikinciye 500 TL transfer edilir.

```java
        //Para transferi
        System.out.println("\n=== Para transferi yapılıyor ===");
        bankaService.hesapAc(musteri, "vadesiz");
        VadesizHesap hesap2=(VadesizHesap) musteri.getHesaplar().get(1);
        bankaService.paraTransferi(hesap, hesap2, 500);
        System.out.println("=== Gönderen hesap ===");
        System.out.println(hesap);
        System.out.println("=== Alan hesap ===");
        System.out.println(hesap2);
```

Şekil 27: Test.java - Para transferi bloğu

- Müşteriye 10000 TL limitli kredi kartı tanımlanır ve 100 TL borç ödenir.

```java
        //Kredi kart tanımlamak
        System.out.println("\n=== Kredi Kart Tamamlanıyor ===");
        bankaService.kredikartTanimla(musteri, 10000);
        KrediKarti kart=musteri.getKrediKartlari().get(0);
        System.out.println(kart);//// Ödeme öncesi kart bilgileri

        //Borc ödemek
        System.out.println("\n=== Borç ödeniyor ===");
        kart.setGuncelBorc(500);
        bankaService.borcOde(hesap, kart, 100);
        System.out.println(kart);// Ödeme sonrası kart bilgileri
```

Şekil 28: Test.java - Kredi kartı tanımlama ve borç ödeme bloğu

- Bakiyesi olan hesap silinmeye çalışılır ve uyarı mesajı gösterilir.

```java
        //Hesap silmek
        System.out.println("\n=== Hesap siliniyor ===");
        bankaService.hesapSil(musteri, hesap2);
```

Şekil 29: Test.java - Hesap silme bloğu

2.4 — Ekran Çıktıları

Aşağıda Main sınıfında gerçekleştirilen işlemlerin ekran çıktıları gösterilmektedir.

1. Müşteri ve Banka Personeli Oluşturma

Müşteri ve banka personeli nesneleri oluşturulmuş ve bilgileri ekrana yazdırılmıştır. Müşteri numarası ve personel ID'si Random sınıfı kullanılarak otomatik olarak üretilmiştir.

```
=== Musteri Olusturuluyor ===
Ad: Zinha, soyad: Tanteliharisoa, email: zinhaantsaharisoa@gmail.com, telefon numarasi: 683007591, MusteriNo: A252604

=== Banka Personeli Olusturuluyor ===
Ad: Ahmet, soyad: Kaya, email: ahmet@btu.edu.tr, telefon numarasi: 5551234, PersonelID: ID20577
```

Şekil 30: Program çıktısı - Müşteri ve banka personeli oluşturma

2. Hesap Açma ve Para Yatırma

Müşteri adına vadesiz hesap açılmış ve hesaba 1000 TL yatırılmıştır. IBAN numarası otomatik olarak üretilmiştir.

```
=== Hesap Aciliyor ===

=== Para Yatiriliyor ===
IBAN: TR616788, Bakiye: 1000.0, Hesap Turu: Vadesiz
```

Şekil 31: Program çıktısı - Hesap açma ve para yatırma

3. Para Transferi

Müşteriye ikinci bir vadesiz hesap açılmış ve birinci hesaptan ikinciye 500 TL transfer gerçekleştirilmiştir. Her iki hesabın bakiyesi 500 TL olmuştur.

```
=== Para transferi yapılıyor ===
=== Gönderen hesap ===
IBAN: TR616788, Bakiye: 500.0, Hesap Turu: Vadesiz
=== Alan hesap ===
IBAN: TR573241, Bakiye: 500.0, Hesap Turu: Vadesiz
```

Şekil 32: Program çıktısı - Para transferi

4. Kredi Kartı Tanımlama ve Borç Ödeme

Müşteriye 10000 TL limitli kredi kartı tanımlanmıştır. Ardından 500 TL borç yüklenmiş ve 100 TL ödeme yapılmıştır. Güncel borç 400 TL'ye düşmüş, kullanılabilir limit ise 9600 TL olmuştur.

```
=== Kredi Kart Tamamlanıyor ===
Kart Numarasi: 787455, Limit: 10000.0, Guncel Borc: 0.0, Kullanilabilir Limit: 10000.0


=== Borç ödeniyor ===
Kart Numarasi: 787455, Limit: 10000.0, Guncel Borc: 400.0, Kullanilabilir Limit: 9600.0
```

Şekil 33: Program çıktısı - Kredi kartı tanımlama ve borç ödeme

5. Hesap Silme

Bakiyesi 500 TL olan hesap silinmeye çalışılmış ve sistem "Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız." uyarısını göstermiştir. Bu sayede bakiyesi olan hesabın silinemeyeceği doğrulanmıştır.

```
=== Hesap siliniyor ===
Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız.
```

Şekil 34: Program çıktısı - Hesap silme

EKSİKLİKLER VE GELİŞTİRMELER

3.1. Grafik Kullanıcı Arayüzü (GUI)

Ne yapılacaktı? → JavaFX veya Swing kütüphaneleri kullanılarak grafik arayüz eklenmesi.
Neden eklenmedi? → Proje şartnamesi sadece konsol uygulaması istediği için.
Eklenirse ne kazandırır? → Daha kullanıcı dostu bir deneyim ve görsel geri bildirim sağlanır.

3.2. Veritabanı Entegrasyonu

Ne yapılacaktı? → Müşteri, hesap ve kredi kartı bilgilerinin bir veritabanına kaydedilmesi.
Neden eklenmedi? → Proje kapsamı basit tutulmak istendi.
Eklenirse ne kazandırır? → Program kapatıldıktan sonra verilerin kaybolmaması sağlanır.

3.3. Daha Fazla Hesap Türü

Ne yapılacaktı? → Mevduat hesabı, döviz hesabı gibi farklı hesap türlerinin eklenmesi.
Neden eklenmedi? → Zaman kısıtı ve proje gereksinimlerinin karşılanması öncelikliydi.
Eklenirse ne kazandırır? → Daha kapsamlı bir banka simülasyonu oluşturulur.

3.4. Hata Yönetimi

Ne yapılacaktı? → Kullanıcı hatalarını yakalamak için try-catch mekanizmalarının eklenmesi.
Neden eklenmedi? → Proje kapsamında zorunlu tutulmadığı için.
Eklenirse ne kazandırır? → Program beklenmedik durumlarda çökmeden devam edebilir.

Proje geliştirme sürecinde en çok kalıtım ve sınıflar arası ilişkilerin kurulması konusunda zorlanılmıştır. Özellikle ArrayList kullanımı ve paket organizasyonu öğretici olmuştur.

SONUÇ

Bu proje kapsamında, Java programlama dilinin nesneye yönelik programlama prensipleri uygulamalı olarak öğrenilmiştir. Kalıtım, kapsülleme, sınıflar arası ilişkiler ve modüler programlama başarıyla gerçekleştirilmiştir.

Projenin en önemli başarısı, şartnamede belirtilen tüm teknik gereksinimlerin eksiksiz olarak karşılanmasıdır. Özellikle:

- **Kalıtım:** Kisi sınıfından Musteri ve BankaPersoneli, BankaHesabi sınıfından VadesizHesap ve YatirimHesabi türetilmiştir.
- **Kapsülleme:** Tüm sınıflarda alanlar private olarak tanımlanmış, getter ve setter metotları yazılmıştır.
- **ArrayList Kullanımı:** Müşteri hesapları ve kredi kartları ArrayList yapısında saklanmıştır.
- **Rastgele Numara Üretimi:** IBAN, müşteri numarası, personel ID ve kart numarası Random sınıfı kullanılarak otomatik olarak üretilmiştir.
- **Paket Organizasyonu:** Sınıflar işlevlerine göre farklı paketler altında organize edilmiştir.
- **Modüler Yapı:** BankaService sınıfı sayesinde iş mantığı ana sınıftan ayrılmıştır.

Sonuç olarak, bu proje hem Java programlama becerilerinin geliştirilmesi hem de nesneye yönelik programlama prensiplerinin uygulamalı olarak öğrenilmesi açısından oldukça verimli olmuştur.

KAYNAKÇA

1. Bursa Teknik Üniversitesi. (2026). Nesneye Yönelik Programlama Proje Ödevi Dokümanı. Bursa Teknik Üniversitesi.
2. Oracle. (2024). Java Documentation. https://docs.oracle.com/en/java/
3. W3Schools. (2024). Java Tutorial. https://www.w3schools.com/java/
4. W3Schools. (2024). Java ArrayList. https://www.w3schools.com/java/java_arraylist.asp